

PARKING DILEMMA

Company: Twitter

Round: Online Assessment

Year: 2021

Difficulty: Medium

Topic: Arrays, Sorting, Sliding Window

Source: https://algo.monster/problems/twitter_oa_parking_dilemma

Problem Description:

Imagine a parking lot where cars are parked at different positions.

You want to build one roof that protects at least k cars from the rain.

The roof should be as small as possible, while still covering at least k cars.

Our task is to find the minimum number of parking slots that the roof needs to cover.

Important Idea

If the roof starts at the position of the first car and ends at the position of the last car, the number of parking slots covered is:

$\text{Last Position} - \text{First Position} + 1$

The $+1$ is important because both the starting and ending parking slots are included.

Input Format:

We are given : pos

An array containing the positions of all the cars.

Example:

pos = [2, 10, 8, 17]

We are given : k

The minimum number of cars that the roof must cover.

k = 3

Output Format:

Return an integer representing the minimum number of parking slots that the roof must cover to protect at least k cars.

Sample Input:

pos = [2, 10, 8, 17]

k = 3

Sample Output:

Minimum Number of Parking Slots = 9

Explanation:

First, sort the car positions:

[2, 8, 10, 17]

We need to cover at least 3 cars.

First 3 cars

2, 8, 10

A roof covering these cars must go from position 2 to position 10.

The number of parking slots covered is:

$$10 - 2 + 1 = 9$$

Next 3 cars

8, 10, 17

The roof would need to cover:

$$17 - 8 + 1 = 10$$

We compare the two possibilities:

9

10

The smaller one is:

9

Therefore, the minimum roof size is:

Approach to Solve This Problem:

The main idea is very simple:

Sort the car positions and check every group of k consecutive cars.

Why do we need to sort?

Because the cars are given in any order.

For example:

[2, 10, 8, 17]

doesn't tell us which cars are physically next to each other.

After sorting:

[2, 8, 10, 17]

we can easily consider cars that are close to each other.

Step 1: Sort the Positions

[2, 10, 8, 17]

becomes:

[2, 8, 10, 17]

Step 2: Consider k Consecutive Cars

If $k = 3$, consider:

[2, 8, 10]

Then:

[8, 10, 17]

Each group contains exactly 3 cars.

Step 3: Calculate the Roof Size

For each group:

roof size = last position - first position + 1

For [2, 8, 10]:

$$10 - 2 + 1 = 9$$

For [8, 10, 17]:

$$17 - 8 + 1 = 10$$

Step 4: Keep the Smallest Answer

We compare all possible groups and keep the smallest roof size.

For this example:

$$\text{minimum}(9, 10) = 9$$

So the answer is 9.

Why Do We Only Check Consecutive Cars?

After sorting, suppose we choose cars at positions:

2, 8, 10

These are three consecutive cars.

If we instead skip a car in between, the roof would still have to cover the skipped car's position if it lies inside the roof.

So choosing consecutive cars gives us the smallest possible range for any group of k cars.

That is why checking every window of k consecutive positions is enough.

Java Code:

```
C:\Users\User\OneDrive\Desktop\Java ID Session\Main.java
1
2
3 public class Main {
4
5     public static int minRoofSize(int[] pos, int k) {
6
7         // Sort the car positions
8         Arrays.sort(pos);
9
10        int minSize = Integer.MAX_VALUE;
11
12        // Check every group of k consecutive cars
13        for (int i = 0; i <= pos.length - k; i++) {
14
15            int first = pos[i];
16            int last = pos[i + k - 1];
17
18            int roofSize = last - first + 1;
19
20            minSize = Math.min(minSize, roofSize);
21        }
22
23        return minSize;
24    }
25 }
```

```
25 |  
26 | Run | Debug  
27 | public static void main(String[] args) {  
28 |     Scanner sc = new Scanner(System.in);  
29 |  
30 |     int n = sc.nextInt();  
31 |  
32 |     int[] pos = new int[n];  
33 |  
34 |     for (int i = 0; i < n; i++) {  
35 |         pos[i] = sc.nextInt();  
36 |     }  
37 |  
38 |     int k = sc.nextInt();  
39 |  
40 |     System.out.println(minRoofSize(pos, k));  
41 |  
42 |     sc.close();  
43 | }  
44 |
```

Solution:

The solution can be summarized in four simple steps:

1. Sort all car positions.
2. Look at every group of k consecutive cars.
3. Calculate the roof size using:

last position - first position + 1

4. Return the smallest roof size found.

For the given example:

pos = [2, 10, 8, 17]

k = 3

After sorting:

[2, 8, 10, 17]

The possible roof sizes are:

$$10 - 2 + 1 = 9$$

$$17 - 8 + 1 = 10$$

Therefore:

Answer = 9

Understanding the Important Part of the Code:

1. Sort the Positions

```
Arrays.sort(pos);
```

This arranges the cars from the smallest position to the largest position.

For example:

```
[10, 2, 17, 8]
```

becomes:

```
[2, 8, 10, 17]
```

2. Check Every Group of k Cars

```
for (int i = 0; i <= pos.length - k; i++)
```

This allows us to examine every possible group of k consecutive cars.

If:

```
n = 4
```

```
k = 3
```

we have two groups:

```
[2, 8, 10]
```

```
[8, 10, 17]
```

3. Find the First and Last Position

```
int first = pos[i];
```

```
int last = pos[i + k - 1];
```

For a group of k cars:

- first is the position of the first car.
- last is the position of the last car.

4. Calculate Roof Size

```
int roofSize = last - first + 1;
```

Suppose:

```
first = 2
```

```
last = 10
```

Then:

$$10 - 2 + 1 = 9$$

The roof covers 9 parking slots.

5. Keep the Minimum

```
minSize = Math.min(minSize, roofSize);
```

Every possible group gives us a roof size.

We keep the smallest one because the problem asks for the minimum roof size.

Complexity Analysis:

Time Complexity: $O(n \log n)$

Sorting the positions takes:

$O(n \log n)$

Then we check all groups of k cars, which takes:

$O(n)$

Therefore, the overall time complexity is:

$O(n \log n)$

Space Complexity: $O(1)$

Apart from the sorting implementation's internal memory, the algorithm itself uses only a few variables such as first, last, roofSize, and minSize.

So the auxiliary space used by the algorithm is $O(1)$.